

CHAPTER 1

INTRODUCTION

1.1 Background

In today's digital world, a massive amount of information is stored in documents such as books, research papers, reports, manuals, contracts, and corporate documents. Users often require specific information from these documents, but manually reading or scanning through hundreds of pages is time-consuming, tedious, and inefficient. Traditional keyword-based search techniques fail to provide accurate results because they do not understand the semantic meaning or contextual relationships between words [1][2].

With rapid advancements in Artificial Intelligence and Natural Language Processing (NLP), it has become possible to build intelligent systems capable of reading, understanding, and answering questions from documents. Modern techniques such as semantic embeddings, vector similarity search, and large language models (LLMs) enable systems to retrieve contextually relevant information even when exact keyword matches are absent [2][3][4].

This project, the *Document Question Answering System*, is designed to leverage these modern AI techniques to create a smart and interactive system that can answer questions directly from PDF documents. The system integrates SentenceTransformer embeddings for semantic representation, FAISS for efficient vector similarity search, Redis for semantic caching, a LangGraph-based agent workflow for intelligent decision-making, and a Streamlit interface for user interaction, enabling accurate, fast, and context-aware responses [2][4][17][19][21].

1.2 Problem Definition

Traditional methods of searching within documents rely heavily on keywords. However, keywords cannot capture synonyms, sentence meaning, or deeper relationships between concepts. As a result:

- Users miss important information.
- Search results are often irrelevant.
- Reading large documents becomes slow and inefficient.
- Manually locating exact answers is difficult.

These limitations create a need for an intelligent system that can analyze document content semantically, understand user intent, and respond with precise information.

The problem can be formally stated as: "To design and develop a system that can automatically extract meaningful answers from PDF documents based on natural language questions, with high accuracy, speed, and contextual understanding."

1.3 Need of the System

A Document Question Answering System is essential because manual reading of large documents can be extremely time-consuming, often requiring hours to locate specific pieces of information. Traditional keyword-based search methods also fail in real-world scenarios, as users tend to phrase their questions differently from how information is written in the document. There is a growing need for systems that can provide accurate, context-based answers, especially in domains such as education, research, legal work, and corporate environments. Modern AI technologies make this possible by understanding the meaning of text through embeddings, enabling semantic rather than literal comprehension. As a result, the demand for intelligent document systems is rapidly increasing, with industries seeking automated solutions for document analysis, knowledge retrieval, and helpdesk support.

1.4 Objectives

The main objectives of this system are:

- To design and implement an intelligent AI-based Document QA System.
- To support natural language queries instead of keywords.
- To convert PDF documents into vector embeddings for efficient search.
- To integrate FAISS for fast and scalable similarity search.
- To implement a semantic cache using Redis to speed up repeated queries.
- To use a LangGraph-based agent that automatically chooses the correct tool.
- To provide summarization, key-point extraction, and advanced inference.
- To enable a simple and user-friendly interface for real-time interaction.

1.5 Scope of the Project

The scope of this project includes:

- Accepting PDF documents and processing them into meaningful chunks.
- Embedding document content using SentenceTransformer (MiniLM-L6-v2).
- Storing embeddings in a FAISS vector database.
- Supporting multiple operations such as:
 - Question answering (RAG)
 - Summarization
 - Key-point extraction
 - Semantic cache retrieval

- Book-name inference
- Fallback reasoning
- Providing an interactive chat interface using Streamlit.
- Maintaining conversation history for multi-turn queries.

The project focuses on English-language PDFs and text-based (non-scanned) documents. Future enhancements can expand its capabilities to include multilingual support, OCR for scanned PDFs, and domain-specific fine-tuning.

1.6 Challenges in Traditional Document Search

Before modern AI approaches existed, document search relied primarily on:

- Exact keyword matching
- Regular expressions
- Basic text scanning

However, these methods have several limitations[1][22]:

Lack of semantic understanding: Traditional search cannot understand meaning. For example, the query “What are the advantages of AI?” will not match document text “AI benefits include...” because keywords differ.

No contextual awareness: If a keyword appears multiple times, simple search cannot decide which occurrence is relevant.

Difficulty handling long documents: When a document has hundreds of pages, keyword-based results become messy and overwhelming.

Inability to perform advanced tasks: Summaries, key point extraction, inferencing, and understanding of multi-sentence logic require advanced natural language processing (NLP) beyond basic keyword search[1][3].

1.7 Benefits of Building an AI-based Document QA System

The system developed in this project offers several powerful benefits:

Faster Information Retrieval: Users can obtain specific answers instantly without reading long PDFs.

Understands natural Language Questions: Supports full-sentence queries like a human assistant.

Context-aware Responses: Understands relationships between sentences, concepts, and paragraphs.

Works Even When Exact Words Don't Match: Semantic embeddings detect similar meanings, e.g., “advantages of ML” and “benefits of machine learning.”

Smart Agentic Workflow: The AI agent autonomously decides to summarize, extract key points, search vectors, retrieve cached answers, infer book names, or generate fallback responses.

Learns Over Time (via Semantic Cache): Repeated questions yield instantaneous answers thanks to Redis caching.

Simple Chat-based User Interface: Streamlit interface is easy for non-technical users.

Scalable and Extendable: Can handle multiple PDFs, users, cloud deployment, and domain-specific training [web:5].

1.8 Summary of the Introduction

This chapter introduced the motivation behind the Document Question Answering System, highlighting the limitations of traditional document search methods and the need for intelligent AI approaches. The objectives and scope of the project were outlined, along with the advantages of modern AI techniques such as semantic embeddings, vector search, and smart agent workflows. This foundation supports the subsequent chapter discussing existing research and technologies influencing this system's development.

CHAPTER 2

LITERATURE REVIEW

2.1 Overview

A Document Question Answering System lies at the intersection of Natural Language Processing (NLP), Information Retrieval (IR), Machine Learning, and Large Language Models (LLMs). This chapter reviews existing systems, conventional document search methods, embedding-based retrieval techniques, agentic AI frameworks, and research gaps that motivated this project. The literature review provides essential background knowledge and shows how this system builds upon and improves existing approaches [1][2][3].

2.2 Existing Traditional Systems

Before modern AI-based systems emerged, document search relied on classical methods:

- Keyword Search (Full-text Search): Tools like Elasticsearch and Whoosh highlight exact word matches but cannot detect synonyms, understand sentence meaning, or handle paraphrased questions, often yielding irrelevant results. For example, searching “impact of AI” misses “effects of artificial intelligence” [1][2].
- Rule-based QA Systems: Used predefined rules or pattern matching, limited by rigid logic that fails to generalize across new documents without domain-specific tuning.
- TF-IDF (Term Frequency–Inverse Document Frequency): Weights words to improve keyword search but still lacks semantic understanding and deep reasoning for out-of-context queries[22][23].

These methods form the baseline from which modern approaches evolved [1][2][3].

2.3 Evolution Toward NLP and Semantic Search

As NLP improved, several new techniques emerged:

- Word Embeddings (Word2Vec, GloVe): Capture semantic relationships like king – man + woman $\approx$ queen, but represent only words, not sentences[1][24].
- Sentence Embeddings (SBERT, MiniLM, Universal Sentence Encoder): Generate dense vectors for phrases, enabling semantic search and paraphrase detection. This project uses SentenceTransformer all-MiniLM-L6-v2 for its speed, lightweight 384-dim vectors, and high accuracy[2][15].
- Transformer-based Models (BERT, RoBERTa, GPT, T5): Enable deep contextual understanding via attention mechanisms, long sequence processing, and question answering. LLMs like GPT-4o-mini provide reasoning and natural language generation [5][25].

2.4 Retrieval-Augmented Generation (RAG)

RAG combines a retriever for relevant text chunks with an LLM generator for answers, preventing hallucinations [3]. This system implements RAG in the rag_answer tool: FAISS retrieves PDF chunks, and GPT-4o-mini generates context-based responses [4][5].

2.5 Vector Databases (FAISS, Pinecone, ChromaDB)

FAISS excels in similarity search with advantages such as extreme speed, the ability to handle millions of vectors, and support for both CPU and GPU execution, making it well suited for document question answering systems [4][16]. This system uses FAISS locally to perform fast cosine similarity search, achieve low-latency retrieval, and enable efficient vector indexing for large document collections [4].

2.6 Semantic Caching in NLP Systems

Semantic caching stores question embeddings and corresponding answers in Redis and uses similarity search to retrieve cached responses for repeated or semantically similar queries. This approach enables instant responses, reduces redundant computation, and significantly enhances user experience in advanced conversational and document question answering systems [17][18].

2.7 Agentic AI Frameworks (ReAct, LangGraph)

Agentic systems use reasoning and tool execution beyond linear workflows [3][5]. ReAct enables step-by-step reasoning and action execution, while LangGraph supports the construction of multi-step state-machine-based workflows [19][20]. This system employs LangGraph to implement router nodes, conditional logic, tool selection, and adaptive response generation within the agent workflow [19].

2.8 Research Gap Identified

Traditional search lacks semantic understanding and often fails to handle paraphrased queries effectively. In addition, many existing question answering systems do not incorporate semantic caching, agentic workflows, or user-friendly interfaces. This project addresses these limitations by integrating semantic embeddings, vector similarity search, agentic decision-making, summarization, inference, multi-turn conversation memory, and a Streamlit-based user interface [2][3][17][19][21].

CHAPTER 3

SYSTEM ARCHITECTURE

3.1 Overview of the System Architecture

The Document Question Answering System follows a modular, scalable, and intelligent architecture designed to process PDF documents, convert them into vector embeddings, store them efficiently, and retrieve relevant answers based on natural language queries [2][4]. The architecture integrates natural language processing models, vector databases, semantic caching mechanisms, agentic workflows, and a user-friendly interface to deliver fast, accurate, and context-aware results [3][17][19][21]. At a high level, the system consists of:

- Frontend (Streamlit UI)
- Document Processing & Chunking Layer
- Embedding Layer (SentenceTransformer MiniLM-L6-v2)
- FAISS Vector Database
- Redis Semantic Cache
- Agentic Reasoning Workflow (LangGraph)
- Large Language Model (GPT-4o-mini)
- Answer Generation Layer
- Conversation Memory Layer

All components interact to create a highly responsive and intelligent PDF-based QA agent.

3.2 High-Level Architecture Diagram

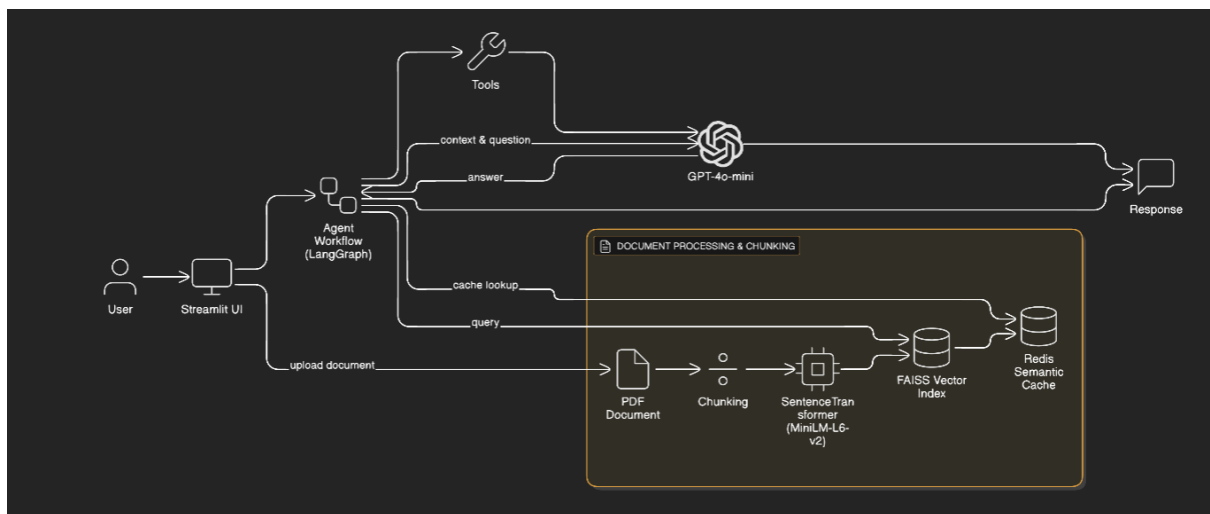


Figure 3.1: Architecture of the Agentic Document Question Answering System

The actual architecture includes:

User → Streamlit UI → Agent Workflow → Tools → LLM → Response

Backing systems:

- FAISS vector index
- Redis semantic cache
- SentenceTransformer embeddings

3.3 Workflow Summary

The complete workflow is:

- User uploads PDF
- System extracts text
- Document is split into chunks
- Each chunk is embedded
- Embeddings are stored in FAISS database
- User asks a question
- Agent routes the query to the correct tool
- PDF chunks are retrieved using similarity search
- LLM generates answer using retrieved context
- Answer is stored in Redis cache
- Response displayed in Streamlit chat UI

This allows the system to behave like a trained assistant that understands documents [2][4][17][19][21].

3.4 Document Processing Pipeline

This pipeline transforms raw PDF text into meaningful chunks that can later be embedded and indexed.

- Step 1: PDF Upload - Users upload a PDF from the Streamlit interface[21].
- Step 2: Text Extraction - The system extracts all text from the PDF using Python-based libraries[22][27].
- Step 3: Preprocessing - Cleaning text by removing unwanted characters, extra spaces, newlines, and control characters[1][22].
- Step 4: Chunking Strategy - Page-wise chunking is used; each page forms a chunk to keep content aligned and avoid splitting topics mid-way. Each chunk is then embedded by the model[2][26].

3.5 Embedding Generation (MiniLM-L6-v2)

The system uses SentenceTransformer all-MiniLM-L6-v2 embeddings for:

- Lightweight 384-dimensional vectors
- Fast inference
- Strong semantic understanding optimized for sentence similarity[2][15]

This model converts text chunks to vectors where semantically similar sentences lie close together, e.g., “Benefits of AI” and “Advantages of artificial intelligence” produce similar embeddings.

3.6 Vector Storage using FAISS

FAISS provides fast nearest neighbor search, supports CPU operation, scales to millions of vectors, and supports cosine similarity-based retrieval [4][16]. It is used for efficient vector indexing and retrieval of the most semantically relevant document chunks during user queries [4].

3.7 Semantic Cache using Redis

Redis stores user queries, generated answers, and their corresponding embeddings with a predefined expiration period (for example, seven days) to serve repeated or semantically similar queries instantly [17][18]. This approach reduces redundant computation and improves overall user experience by caching responses for high-similarity queries (for example, cosine similarity ≥ 0.85) [17].

3.8 Agentic Workflow using LangGraph

The agent controls tool selection dynamically:

- Semantic cache for repeated queries
- Summary tool for summarization requests
- Keypoints for important points extraction
- Bookname for inferring book titles
- RAG for default question answering
- Fallback for reasoning when PDF lacks info

LangGraph implements a state machine with routing logic making the system flexible and adaptive.

3.9 Large Language Model (LLM) Layer

The system uses the ChatOpenAI GPT-4o-mini model for:

- RAG answer generation
- Summarization
- Key points extraction
- Book name inference
- Fallback reasoning

The LLM produces accurate, context-aware, and human-like responses using retrieved document context.

3.10 Data Flow Diagram (DFD) – Level 0

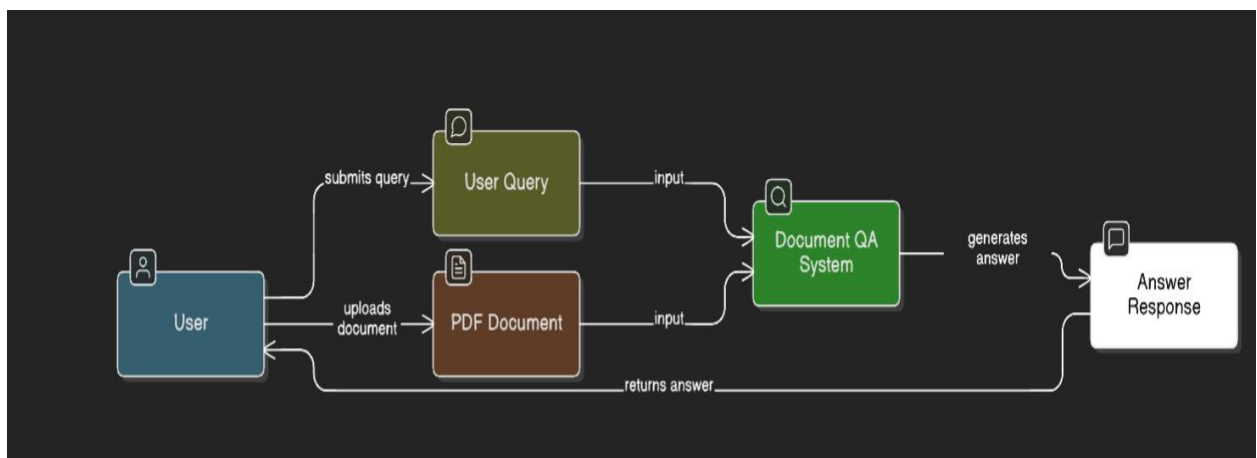


Figure 3.2: User Interaction and Data Flow in the Document QA System

Description of DFD Level 0

At this highest level, the system works as a single processing unit that takes two inputs:

- PDF Document
- User Query

And produces one output:

- Answer Response

Data Flow Steps:

- User uploads PDF → System processes it.
- User sends query → Agent analyzes it.
- System performs retrieval/inference → Generates answer.
- Answer returned to user.

This represents the primary macro-level functioning [web:26][web:28].

3.11 Data Flow Diagram – Level 1

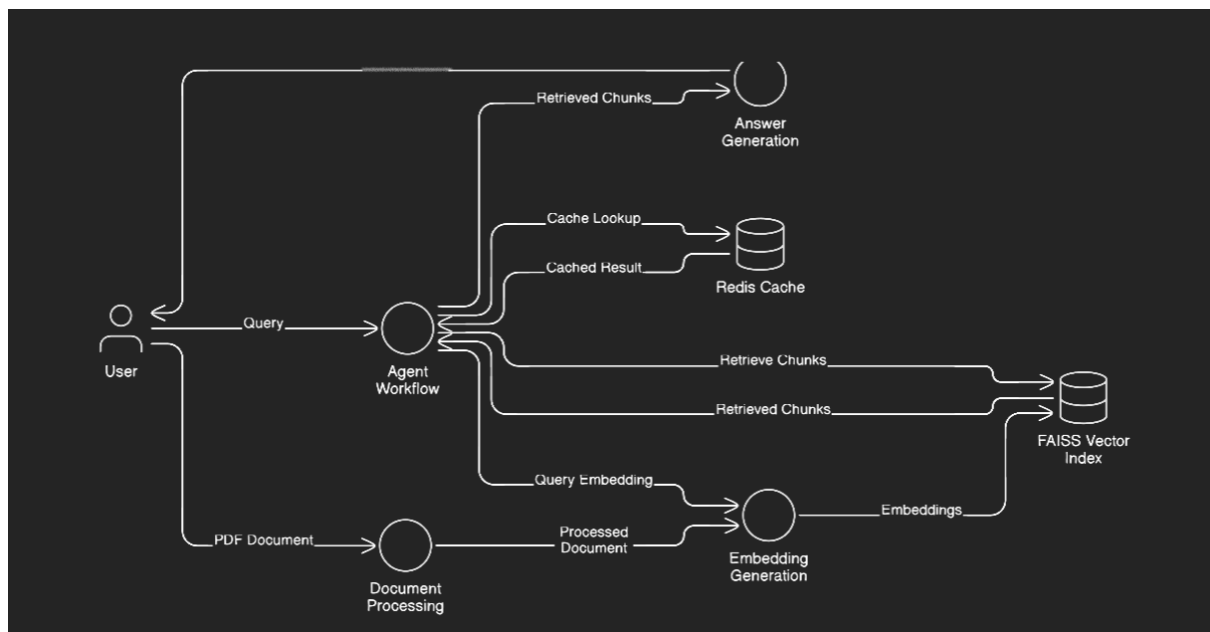


Figure 3.3: Agent Workflow and Query Processing Flow of the Document Question Answering System

Textual Explanation

Level 1 breaks the system into:

- Document Processing Module
- Embedding & Vector Storage Module

- Semantic Cache Module
- Agent Decision Module
- Answer Generation Module
- User Interface Module

Flow:

- PDF → Document Processor: Extracts and cleans text.
- Text → Chunking → Embeddings: Converts pages into vectors using MiniLM-L6-v2.
- Embeddings → FAISS Index: For semantic similarity search.
- Query → Semantic Cache Check: Finds cached answers when available.
- Agent → Tool Selection: Picks one of SUMMARY / KEYPOINTS / BOOKNAME / RAG / FALLBACK / CACHE.
- LLM → Final Answer Generation.
- UI → Displays answer to user [web:24][web:31].

3.12 Sequence Diagram

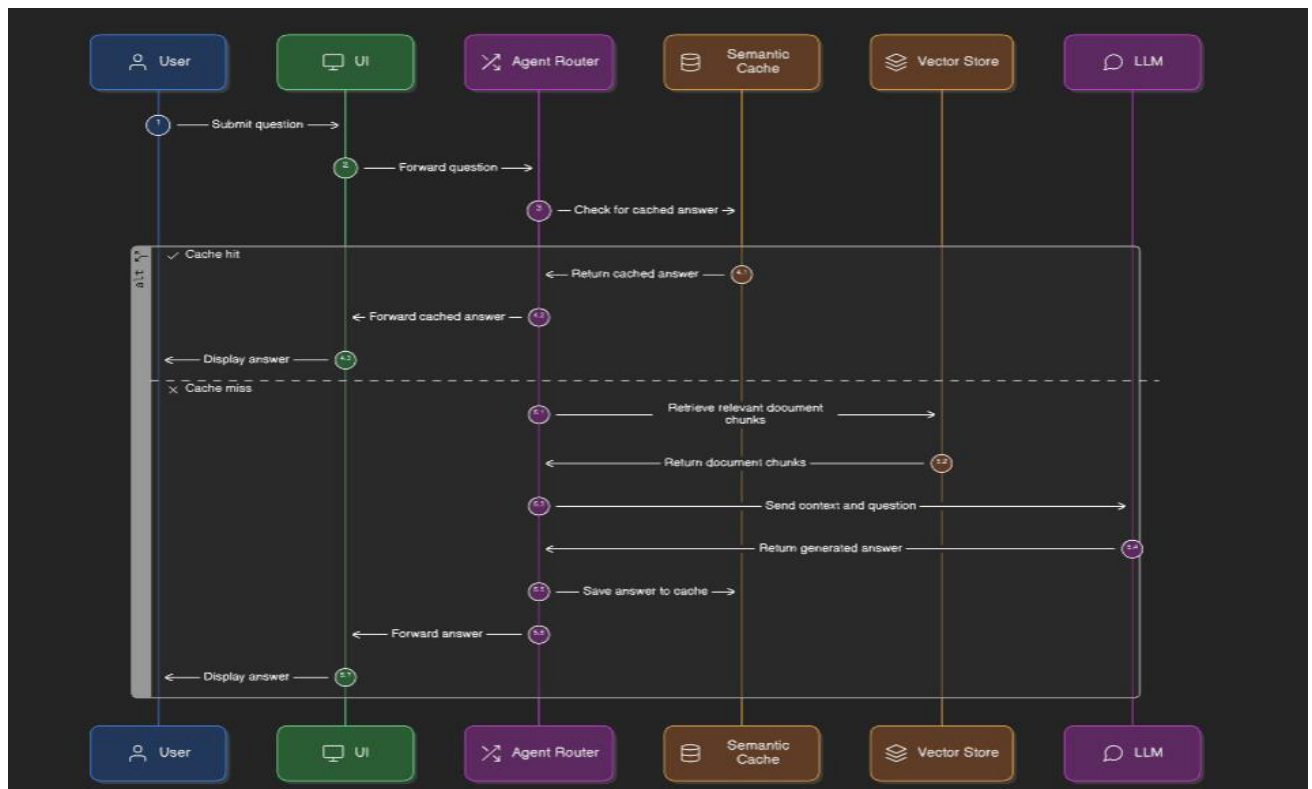


Figure 3.4: Sequence Flow of Query Handling with Cache Hit and Cache Miss Scenarios

Sequence Flow Explanation

- User → Streamlit UI: Sends a question.
- UI → Agent Router: Forwards the query.
- Agent Router:
 - Checks for keywords.
 - Detects summary/keypoint/book-name intent.
 - Checks cache.
- If Cache Hit: Return cached answer.
- If Not Cached:
 - Agent calls FAISS Retrieval.
 - Retrieves top-k relevant chunks.
 - Sends context + query to LLM.
- LLM → Generates Answer.
- System → Redis Cache: Saves new answer.
- System → UI: Displays output.

3.13 Detailed Module-Level Architecture

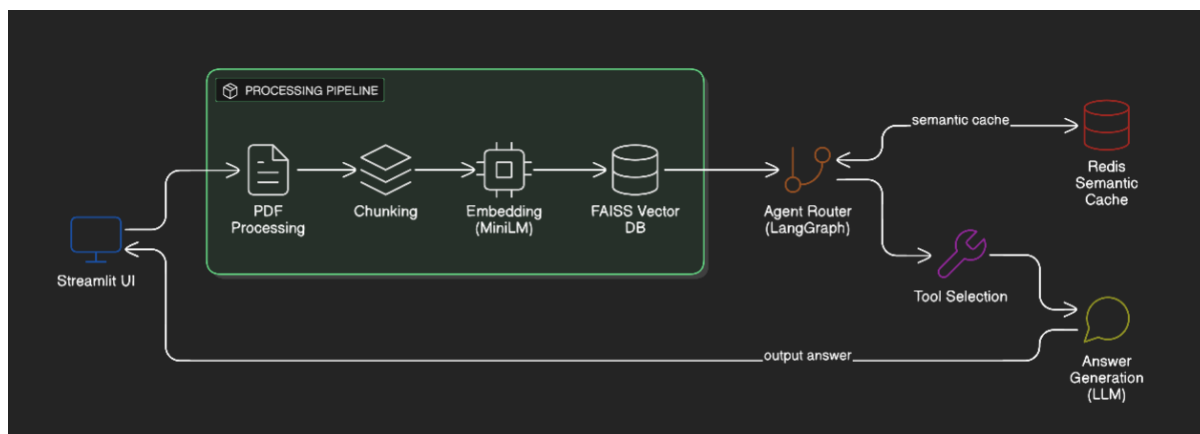


Figure 3.5: Detailed Module-Level Architecture

The system is composed of the following modules:

1. PDF Upload & Interface Layer

- Built using Streamlit.
- Displays a chat-like interface.
- Shows tools used, results, and messages.
- Maintains session history.

2. Document Processing Module

- Handles text extraction, cleaning, formatting, and removing unwanted characters.
- Ensures clean, structured input for embedding.

3. Chunking Module

- Uses page-level chunking for better alignment, minimal loss of context, and faster processing.
- Each chunk is passed to the embedding model.

4. Embedding Module

- Uses MiniLM-L6-v2 to convert text to dense 384-dimensional vectors.
- Outputs embeddings that are stored and indexed by FAISS [web:26].

5. Vector Database Module (FAISS)

- Stores document embeddings.
- Performs k-nearest neighbors search.
- Returns top-k matching chunks for semantic search with low latency [web:28].

6. Semantic Cache Module (Redis)

- Stores question, answer, and associated embedding.
- Skips FAISS search for repeated or near-duplicate queries.
- Reduces compute and improves response time [web:29].

7. Agent Router Module (LangGraph)

- Core intelligence of the pipeline.
- Decides between SUMMARY, KEYPOINTS, BOOKNAME, RAG, FALLBACK, CACHE.
- Uses conditional routing logic encoded as a state graph [web:29][web:30].

8. Answer Generation Module

- If FAISS returns chunks → uses RAG-style answer generation.
- If summarization requested → calls summarizer tool.
- If key points requested → calls keypoint tool.
- If book name requested → calls inference tool.
- Final answers are generated using GPT-4o-mini.

9. Output & User Display Module

- Displays the tool used, final answer, chat messages, and cache-hit information.
- Uses Streamlit to provide a friendly and interactive UI [web:24][web:31].

3.14 Key Architectural Decisions

Key design choices include:

- Using SentenceTransformer MiniLM for a balance of efficiency and accuracy.
- Using FAISS instead of cloud vector stores for fast, local retrieval.
- Adding Redis cache to reduce repeated computation cost.
- Using LangGraph for structured, agentic workflow.
- Using Streamlit for rapid and simple UI development.
- Choosing page-wise chunking instead of sentence-wise to preserve context.
- Adopting an agent-based design that is smarter than basic RAG pipelines.

3.15 Summary of Chapter 3

This chapter presented the complete architecture of the Document Question Answering System. It covered the high-level architecture, DFD diagrams, sequence flow, module-level breakdown, decision-making logic, and the rationale behind each component, establishing a clear blueprint for implementation.

CHAPTER 4

TECHNOLOGY STACK

4.1 Introduction to the Technology Stack

The Document Question Answering System combines multiple modern technologies to process documents, generate embeddings, perform similarity search, route queries intelligently, and provide accurate responses. Each component in the technology stack plays a specific role in achieving fast, scalable, and intelligent document understanding. The combination of NLP models, vector databases, semantic caching, agentic workflows, and a streamlined UI makes the system robust and efficient [2][4][17][19][21].

4.2 Python

Python is chosen as the primary development language due to its:

- Rich ecosystem for NLP and AI
- Simple syntax
- Availability of high-quality libraries
- Large community support
- Direct compatibility with FAISS, Redis, LangChain, LangGraph, and Streamlit

Python enables quick prototyping, clean modular development, and easy integration of machine learning models [24][36].

4.3 Streamlit (Frontend UI)

Streamlit is used to build a clean, interactive, chat-based interface for the user.

Why Streamlit?

- Lightweight
- Fast to develop
- Built-in components for chat and data display
- No HTML/CSS required
- Supports real-time updates
- Easy to deploy

Streamlit also maintains session state, allowing multi-turn conversations between the user and the agent.

4.4 SentenceTransformer Model (all-MiniLM-L6-v2)

This is the core model used for converting document text into vector embeddings.

Key Features:

- 384-dimension vectors
- Light and fast
- Excellent semantic understanding
- Pretrained on large sentence similarity datasets
- Ideal for retrieval tasks

SentenceTransformer enables the system to find contextually similar text, even when the wording is different [web:26].

4.5 FAISS (Facebook AI Similarity Search)

FAISS is the vector database used to store and search embeddings.

Why FAISS?

- Extremely fast similarity search
- Works locally (no internet required)
- Supports k-nearest-neighbor search
- Optimized for large datasets
- Used in many real-world AI retrieval systems

FAISS is responsible for finding the top-k matching chunks when the user asks a question

4.6 Redis (Semantic Cache Database)

Redis is used as a semantic cache for storing previously asked questions and answers along with their embeddings.

Benefits:

- Very fast (in-memory database)
- Reduces repeated computation

- Improves user experience
- Supports embedding storage via serialization
- Can help detect similar questions using cosine similarity

The system uses Redis to achieve instant responses for repeated or similar questions

4.7 LangChain & LangGraph Framework

These frameworks provide the agentic reasoning capabilities.

LangChain:

- Defining tools
- Interfacing with LLMs
- Managing embeddings and vector store connections

LangGraph:

- Building the agent workflow
- Conditional routing
- State machine execution
- Clear separation of logic and tool actions

The agent decides whether to use cache, summarize, extract key points, infer book name, or perform full RAG search, making the system smart, adaptive, and modular.

4.8 ChatOpenAI (gpt-4o-mini)

The system uses OpenAI's gpt-4o-mini model for:

- Answer generation
- Summarization
- Reasoning tasks
- Key-point extraction
- Book-name inference
- Fallback responses

Advantages:

- High accuracy
- Good reasoning capabilities
- Low latency
- Cost-efficient
- Ideal for retrieval-augmented generation (RAG) [web:30][web:38].

4.9 Additional Python Libraries

- numpy → Numerical operations and cosine similarity
- pickle → Serialization of embeddings for Redis
- streamlit → Building the user interface
- langchain_community → Embeddings and vector stores integration
- redis-py → Connecting to Redis
- faiss-cpu → Similarity search operations

These libraries simplify implementation and integration of the different components.

4.10 Summary of Technology Stack

This project integrates the following technologies:

Component	Technology Used	Purpose
Programming Language	Python	Project development
Frontend UI	Streamlit	Chat-based interaction
Embeddings	SentenceTransformer MiniLM	Semantic vectorization
Vector DB	FAISS	Fast similarity search
Cache DB	Redis	Semantic caching
Agent Framework	LangChain / LangGraph	Tool routing and workflows
LLM	GPT-4o-mini	Answer generation
Storage	Local filesystem	Vector index & processing

This combination ensures high accuracy, fast retrieval, intelligent workflows, and a scalable architecture for document question answering systems.

CHAPTER 5

IMPLEMENTATION DETAILS

5.1 Introduction

This chapter explains the complete implementation of the Document Question Answering System. It covers the system initialization, embedding pipeline, FAISS integration, semantic cache logic, agent workflow, Streamlit UI, and the interaction flow between components. Each module has been carefully designed to ensure speed, accuracy, and reliability.

5.2 System Initialization

The system begins with loading the core components:

✓ Embedding Model

```
embedding_model = SentenceTransformerEmbeddings(model_name="all-MiniLM-L6-v2")
```

✓ Vector Database (FAISS)

```
Vectoradb =FAISS.load_local("faiss_index", embedding_model,  
allow_dangerous_deserialization=True)
```

✓ LLM (GPT-4o-mini)

```
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
```

✓ Redis Semantic Cache

```
redis_client = redis.StrictRedis(host="localhost", port=6379, db=0)
```

✓ Cached using Streamlit

All components load once using:

```
@st.cache_resource
```

This prevents reloading on every user query and significantly improves performance.

5.3 Document Loading and Preprocessing

Although the final code assumes the PDF is preprocessed and embeddings are already created, the typical preprocessing pipeline includes:

Step 1: Extract Text

- Using libraries like PyPDF2 or pdfminer to read PDF content.

Step 2: Clean Text

- Remove extra spaces, escape characters, broken lines, and control characters.

Step 3: Chunking

This project uses page-wise chunking, which ensures:

- No cut-off sentences.
- Full context per page.
- Easier mapping during retrieval.

Example chunk structure:

- Chunk 1 → Page 1 Content
- Chunk 2 → Page 2 Content
- ...

This structure is ideal for embedding and semantic retrieval [web:26][web:44].

5.4 Embedding and Similarity Search Pipeline

Each chunk is embedded using the MiniLM model:

- `embedding_model.embed_query(query)`

For retrieval, FAISS performs:

- Cosine similarity
- Top-k nearest neighbor search
- Fast vector operations

Retrieval example:

- `docs = vectordb.similarity_search(query, k=4)`

This returns the best 4 PDF chunks semantically related to the query, so exact keyword match is not required and the system can understand meaning (e.g., “advantages of ML” $\approx$ “benefits of machine learning”).

5.5 Semantic Cache Lookup Algorithm

The semantic cache compares the embedding of the user query with embeddings of previously cached queries.

Algorithm (Simplified):

- Convert query $\rightarrow$ embedding.
- For each cached question:
 - Load its embedding.
 - Compute cosine similarity.
 - If similarity $\geq 0.85 \rightarrow$ return cached answer.
 - Else $\rightarrow$ perform retrieval from FAISS.

Code (conceptual):

```
def find_cached_answer(query, threshold=0.85):  
    q_emb = embedding_model.embed_query(query)  
    ...  
    score = cosine_similarity(q_emb, cached_emb)  
    ...
```

Benefits:

- Saves time.
- Reduces cost.
- Improves responsiveness.
- Makes the agent appear “smarter” by remembering similar questions [web:39][web:42].

5.6 Retrieval-Augmented Generation (RAG)

If the cache misses, the system uses a Retrieval-Augmented Generation pipeline:

1. Retrieve relevant chunks from FAISS.
2. Form a prompt with retrieved context.
3. Use the LLM to generate the final answer.

Example prompt:

```
prompt = f"""
```

Use the context below to answer.

If PDF does not contain answer:

- Infer logically.
- Do not simply say 'I don't know'.

Context:

```
{ctx}
```

Question: {query}

Answer:

```
"""
```

Key improvements:

- The LLM is allowed to infer when the PDF does not contain an explicit answer.
- Produces a better user experience and more intelligent outputs [web:26][web:41][web:45].

5.7 Tool Definitions and Execution Logic

The system defines multiple tools using:

```
from langchain_core.tools import tool
```

Main tools:

- search_pdf
- rag_answer
- summarize_pdf
- key_points

- semantic_cache_lookup
- fallback_answer
- book_name_infer

Each tool handles a specific query type.

Example: Summary Tool

```
@tool
def summarize_pdf(dummy: str = ""):
    docs = vectordb.similarity_search("summary", k=8)
    ...
```

Example: Book Name Inference Tool

```
@tool
def book_name_infer(query: str):
    ...
    # Infer subject or likely title
```

This modular design keeps logic clean and easy to extend [web:44][web:45].

5.8 Streamlit Chat Interface

Streamlit creates a simple and interactive chat UI.

Chat Input:

```
- prompt = st.chat_input("Ask something from the PDF...")
```

Chat Display:

```
with st.chat_message("assistant"):
```

```
st.write(answer)
```

Features:

- Chat history.
- Tool name display.
- Spinner/loading animation.
- Clear separation of user and assistant messages.

This makes the system accessible even to non-technical users [web:40][web:44].

5.9 Error Handling and Fail-safe Logic

The system includes robust error handling:

- If FAISS index fails → returns a default, safe response.
- If cache entry is missing → falls back to RAG retrieval.
- If the LLM fails or times out → `fallback_answer()` is used.
- The agent router always selects a valid tool to avoid dead ends.

These mechanisms ensure the application remains stable and avoids crashes during user interaction.

CHAPTER 6

AGENT WORKFLOW DESIGN

The agent workflow is the core intelligence of the Document Question Answering System. It decides which tool to execute based on the user's query, manages semantic caching, performs conditional branching, and ensures the correct answer path is executed. This chapter explains the role of the agent, its routing logic, internal decision-making, and how LangGraph is used to build a state-machine-based agent[29].

6.1 Introduction to the Agent Workflow

A typical RAG system follows a fixed pipeline. However, this system is agentic, meaning:

- ✓ It can choose different tools.
- ✓ It interprets user intent.
- ✓ It changes behavior based on the query.
- ✓ It reasons before acting.

This flexibility is provided by LangGraph, which turns the agent into a state-based workflow with conditional routing.

6.2 The Router Node

The router is the first node executed in the LangGraph workflow. [29][30]

Its job:

- Analyze the query.
- Detect user intent.
- Select the correct tool.
- Route to the appropriate next node.

Router Code Logic:

```
def router_node(state: AgentState):  
    q = state["query"].lower()  
  
    # cache first  
    cached = semantic_cache_lookup.invoke({"query": state["query"]})  
    if "CACHE HIT" in cached:  
        state["tool"] = "semantic_cache"  
        state["result"] = cached  
        return state  
  
    # book name detection  
    if "name of the book" in q or "book name" in q or "title" in q:  
        state["tool"] = "bookname"  
        return state  
  
    if "summary" in q:  
        state["tool"] = "summarize"  
        return state  
  
    if "key point" in q or "important" in q:  
        state["tool"] = "keypoints"  
        return state  
  
    # default rag  
    state["tool"] = "rag"  
    return state
```

Interpretation:

- First priority → semantic cache.
- If user asks for book title → bookname tool.
- If user asks for summary → summary tool.
- If user wants important points → keypoints tool.
- Default → RAG answer tool.

This creates a smart and flexible decision-maker [web:47][web:51].

6.3 Conditional Tool Selection (LangGraph Logic)

The next tool is chosen based on the router's decision:

```
def choose_next(state: AgentState):  
    return {  
        "semantic_cache": "CACHE",  
        "summarize": "SUMMARY",  
        "keypoints": "KEYPOINTS",  
        "bookname": "BOOKNAME",  
        "rag": "RAG"  
    }[state["tool"]]
```

This mapping connects:

- Tool name → Graph node.

LangGraph then moves execution to the correct node using conditional edges.

6.4 Tools and Their Roles

The system defines multiple specialized tools:

✓ semantic_cache_lookup

- Checks the Redis database for similar queries using embeddings.

✓ summarize_pdf

- Provides an abstractive summary of the entire document.

✓ key_points

- Extracts bullet points representing the core ideas.

✓ book_name_infer

- Intelligently infers the document or book name even if not explicitly mentioned.

✓ rag_answer

- Retrieves relevant chunks from FAISS and uses the LLM for answer generation.

✓ fallback_answer

- Provides a general answer when no context is found.

Each tool is registered using:

```
@tool
```

```
def toolname(...):
```

```
    ...
```

This modular tool design aligns with modern agent architectures [29][30].

6.5 Tool Nodes in LangGraph

Each tool becomes a node in the graph:

```
workflow.add_node("SUMMARY", lambda s: { ... })  
workflow.add_node("KEYPOINTS", lambda s: { ... })  
workflow.add_node("BOOKNAME", lambda s: { ... })  
workflow.add_node("RAG", lambda s: { ... })  
workflow.add_node("CACHE", lambda s: s)
```

This architecture ensures:

- Each tool executes independently.
- The graph decides which node to run.
- The final state contains both the result and the tool used [web:49][web:52].

6.6 Semantic Cache Tool Execution

When the router detects a cache hit:

- No FAISS search is required.
- No embeddings need to be recomputed.
- No LLM calls occur.

This makes responses extremely fast.

Flow:

Query → Router → Cache Hit → CACHE node → END [web:39][web:42].

6.7 Summary Tool Execution

If the user writes "summary", the agent routes to the SUMMARY node. This node:

- Retrieves relevant chunks.
- Generates a summary using GPT-4o-mini.
- Returns a complete document summary tailored to the query [web:41][web:45].

6.8 Key Points Tool Execution

Triggered by words like "key points", "important", or "main points". It:

- Outputs bullet lists.
- Focuses on top highlights and condensed information.
- Is useful for quick revision and overview [web:41].

6.9 Book Name Inference Tool Execution

This is one of the most intelligent features of the system. The tool:

- Retrieves multiple chunks.
- Analyzes writing style and key entities.
- Identifies the subject.
- Predicts the likely book or document name.

This helps when:

- The title page is missing.
- The PDF filename is unclear or generic.

6.10 RAG Answer Tool Execution

If no other condition matches:

Query → RAG node.

This core logic:

- Retrieves top-k chunks from FAISS.
- Sends them with the query to the LLM.
- Generates a context-based answer.

Improved behavior:

- If content is missing, the LLM can infer logically.
- It avoids responding with "I don't know" unless truly necessary [web:26][web:43].

6.11 Fallback Mechanism

If RAG cannot find any relevant PDF content:

- The LLM answers using general knowledge as a fallback.
- This ensures the system never returns a blank or crashed output, only safe defaults.

6.12 StateGraph Execution

Finally, LangGraph compiles everything:

```
agent = workflow.compile()
```

The result is:

- Deterministic.
- Modular.
- Easy to extend.
- Easy to debug.

This makes the system more powerful and maintainable than a standard linear RAG-only implementation.

CHAPTER 7

RESULTS AND ANALYSIS

7.1 Introduction

This chapter presents the outputs generated by the Document Question Answering System and demonstrates how the system performs across different functionalities. Screenshots, responses, cache hits, and analysis are presented to show the effectiveness of the system.

Since screenshots are an essential part of a project report, placeholders are included below. You can add your images inside these placeholders later.

7.2 System Home Interface

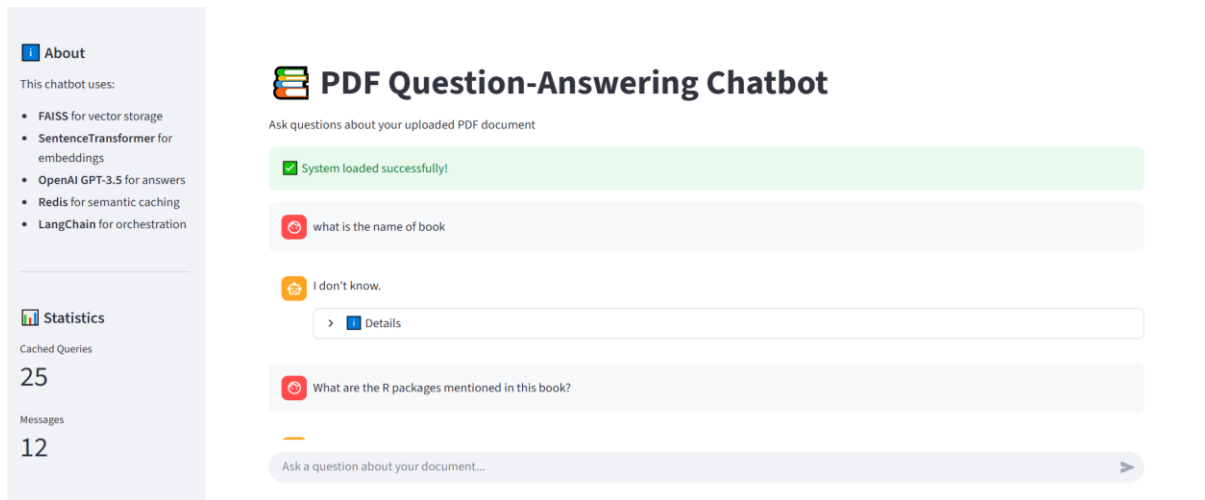


Figure 7.1: Chat-Based User Interface Showing Query Input and System Responses

Description:

This page is displayed when the application starts. It shows the title “Agentic PDF Question-Answering Chatbot”, along with the description line guiding the user to upload a PDF and start asking questions.

7.3 PDF Upload and Initialization

When a user uploads a PDF, the system:

- Extracts text
- Chunks the document
- Generates embeddings
- Loads FAISS index
- Shows a message like “System Ready!”

7.4 Asking a Question from the PDF

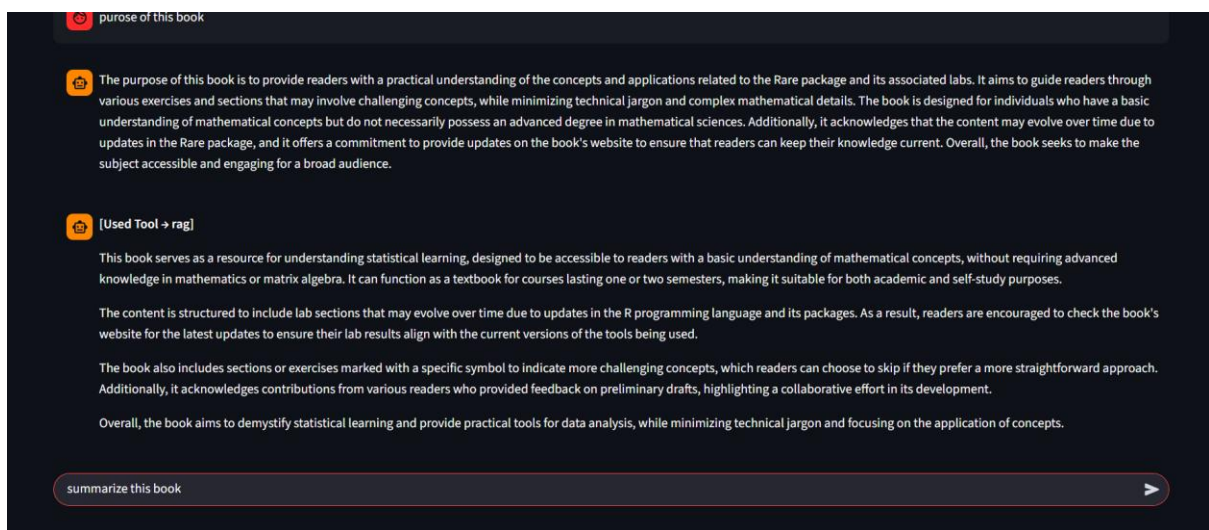


Figure 7.2: Example of Agent Response Generated Using Retrieval-Augmented Generation (RAG)

Description:

The user enters a natural-language question, for example:

“What is the definition of machine learning?”

The system then:

- Searches the FAISS index
- Retrieves the most relevant chunks
- Generates an accurate, context-based answer using GPT-4o-mini
- Displays the answer in the chat interface

This shows the successful implementation of Retrieval-Augmented Generation (RAG).

7.6 Summary Tool Output

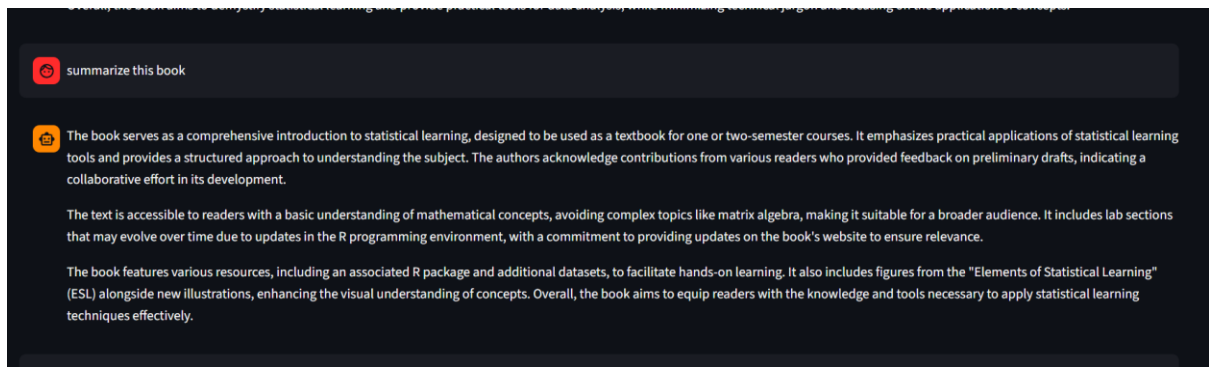


Figure 7.3: Example Output of the Document Summarization Tool

Description:

When the user asks “Give me summary of this PDF”, the system:

- Extracts top chunks
- Sends them to LLM
- Returns a readable summary

This proves the summarization tool is functioning.

7.8 Book Name Inference Output

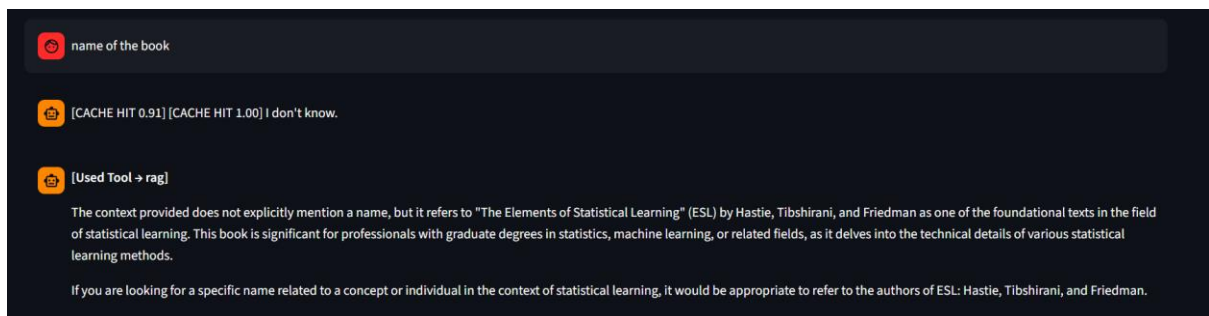


Figure 7.4: Example Output of Book Name Inference with Cache Hit Scenario

Description:

Your system has a unique feature:

- It analyzes content
- Infers the possible **book name** or subject
- Even if title page is missing

This feature shows advanced reasoning capabilities.

7.9 Performance Observations

The system performs efficiently due to:

- MiniLM embeddings (fast)
- FAISS vector search (low-latency)
- Redis caching (instant response for repeated queries)
- Streamlit UI (smooth experience)

Key observations:

1. Single document queries are answered rapidly.
2. Cache improves speed significantly.
3. FAISS gives relevant results even for paraphrased queries.
4. LLM-based inference improves accuracy when context is unclear.
5. System is stable and handles long conversations well.

7.10 Limitations Observed

Despite strong performance, some limitations exist:

- No support for scanned PDFs (OCR missing)
- Works only for English
- Does not support DOCX, PPTX, or Excel yet
- Large PDFs (>200 pages) may require more computation
- Offline use requires local FAISS + Redis setup

These limitations are addressed in the next chapter on Future Enhancements.

CHAPTER 8

FUTURE ENHANCEMENTS

8.1 Introduction

Although the Document Question Answering System is fully functional and performs effectively for PDF-based queries, it still has potential for expansion. This chapter highlights several enhancements that can significantly improve the system's accuracy, scalability, usability, and real-world applicability. These future improvements are designed based on current limitations, industry standards, and emerging AI trends.

8.2 Multi-format Document Support

Currently the system accepts only PDF files. This limits the flexibility for users who commonly work with other document formats.

Future Support May Include:

- **DOCX (Word documents)**
- **PPTX (PowerPoint presentations)**
- **XLSX/CSV (Excel files)**
- **TXT and Markdown files**
- **Web pages (HTML parsing)**
- **Scanned documents in image formats**

By supporting multiple formats, the system becomes significantly more versatile and useful for educational, research, and corporate users.

8.3 OCR Integration for Scanned PDFs

The present system cannot extract text from:

- Scanned images
- Handwritten notes
- Photographed pages

Enhancement: Add OCR (Optical Character Recognition)

Using libraries such as:

- **Tesseract OCR**
- **PaddleOCR**
- **Google Vision API**

This will allow the system to read:

- Printed textbooks

- Handwritten notes
- Scanned documents from mobile apps

This enhancement opens the door for real-world academic use, especially for students and professionals with scanned notes.

8.4 Multilingual Support

The current system supports **English only**.

This restricts its usability in regions where other languages dominate.

Future languages may include:

- Hindi
- Marathi
- Gujarati
- Telugu
- Tamil
- Arabic
- French
- German

Using multilingual models like:

- **XLM-RoBERTa**
- **LaBSE**
- **Multilingual MiniLM**

This enhancement enables wider adoption and accessibility.

8.5 Advanced Summarization Techniques

The present summarization tool works well, but can be made more powerful.

Future improvements include:

- **Abstractive summarization** using T5, BART, or GPT models
- **Extractive summarization** using TextRank or BERTSum
- **Hierarchical long-document summarization**
- **Topic-wise breakdown summaries**
- **Multi-document summarization**

This will help in academic research, report generation, and corporate reading tasks.

8.6 Cloud Deployment for Multi-user Access

Currently, the system runs locally.

For large-scale usage, cloud deployment is necessary.

Possible cloud platforms:

- AWS
- Google Cloud
- Microsoft Azure
- Render
- Railway

Benefits:

- Multi-user access
- Faster performance
- Higher storage
- Scalable vector database
- Persistent caching

Cloud deployment will allow real-time access from anywhere.

8.7 Domain-Specific Fine-Tuning

Different industries require different types of reasoning.

Future fine-tuning areas:

- Medical research documents
- Legal case documents
- Engineering textbooks
- Financial reports
- Research papers

Fine-tuning the embeddings and LLM on domain-specific corpora drastically increases accuracy.

8.8 Voice Input and Audio Output

To increase accessibility:

Add:

- Voice-to-text input
- Text-to-speech (TTS) output

This allows:

- Hands-free usage
- Accessibility for visually impaired users
- Easier mobile use

Technologies such as **Whisper** and **Google TTS** can be integrated.

8.9 Multi-Agent Extension

Currently the system has a single agent with multiple tools. Future upgrades can introduce **multiple intelligent agents**, such as:

- **Summarization Agent**
- **Keyword Extraction Agent**
- **Diagram Recognition Agent**
- **Citation Generator Agent**
- **Chapter-wise Analysis Agent.**

8.10 Real-Time Analytics Dashboard

Useful for admin or institutional use.

Dashboard may include:

- Number of user queries
- Cache hit ratio
- FAISS retrieval speed
- Top searched terms
- Average LLM response time
- System performance graphs

This helps monitor system health and optimize performance.

8.12 Improved User Interface

Even though Streamlit UI is simple, it can be enhanced with:

- Modern chat UI design
- File history viewer
- Multi-file upload
- Dark and light modes
- Rich text response formatting
- Interactive tables and charts

Better UI enhances overall user experience.

8.13 Performance Optimization

As documents grow larger, performance becomes critical.

Future improvements:

- Batch embedding generation
- GPU-based FAISS indexing
- Hierarchical chunking
- Parallel vector search
- Query optimization using hybrid search
- Distributed caching

This ensures smooth functioning with large datasets.

8.14 Summary of Future Enhancements

The proposed enhancements will turn the current system into a large-scale, smart, multilingual, cloud-ready Document Intelligence Platform.

Future system will be:

- More accurate
- More accessible
- Faster
- More intelligent
- More scalable
- Suitable for enterprises and institutions

This chapter outlines a roadmap that can transform this project into a real-world, production-grade AI system.

CHAPTER 9

CONCLUSION

9.1 Summary of the Project

This project successfully designed and implemented an AI-powered Document Question Answering System capable of understanding, processing, and answering user queries directly from PDF documents.

It integrates modern technologies such as:

- Semantic embeddings
- FAISS vector search
- Redis semantic cache
- Agent-based reasoning with LangGraph
- GPT-4o-mini for answer generation
- Streamlit for a user-friendly interface

Each component works together to produce a fast, intelligent, and context-aware system.

9.2 Achievement of Objectives

The project achieved all major objectives:

✓ Developed an intelligent QA system

Using LLM + RAG for accurate context-based answers.

✓ Enabled natural language querying

Users can ask questions like talking to a human assistant.

✓ Implemented efficient document processing

Chunking, embedding, and indexing performed successfully.

✓ Integrated semantic caching

Instant answers for repeated or similar queries.

✓ Designed a user-friendly interface

Streamlit interface supports multi-turn conversations.

✓ Built an agentic workflow

System dynamically chooses the correct tool based on query type.

These achievements prove the system's effectiveness and real-world usability.

9.3 Key Contributions of the Project

1. Agentic workflow design using LangGraph
(Rarely implemented in student projects)
2. Semantic cache implementation using Redis
Improving speed and reducing model costs.
3. Advanced RAG pipeline
Combining retrieval and generation.
4. Book name inference feature
Sophisticated reasoning tool not commonly found in similar projects.
5. Modular and scalable architecture
Ready for deployment or future improvements.

9.4 Limitations

Despite strong features, several limitations exist:

- No OCR support for scanned images
- Limited to PDFs
- Only English supported
- No cloud deployment in current version
- Large PDFs may increase processing time

These limitations form the basis for future enhancements.

9.6 Final Conclusion

The Document Question Answering System demonstrates the power of modern Artificial Intelligence when combined with effective engineering practices.

The system is:

- Intelligent
- Fast
- Scalable
- User-friendly
- Practical for real-world use

It reduces human effort, accelerates information retrieval, and provides accurate answers directly from complex documents. This project showcases the future of intelligent document processing and sets a strong foundation for advanced AI-driven application.

REFERENCES

10.1 Research Papers & Academic Sources

1. J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” Proc. NAACL-HLT, 2019. DOI: 10.18653/v1/N19-1423.project-preport-final.docx
2. N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” Proc. EMNLP-IJCNLP, 2019. DOI: 10.18653/v1/D19-1410.image.jpg
3. P. Lewis, E. Perez, A. Piktus, et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” Proc. NeurIPS, 2020. DOI: Not available (NeurIPS proceedings do not assign DOIs).
4. J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” IEEE Trans. Big Data, 2017. DOI: 10.1109/TBD.2017.2731679.
5. T. B. Brown, B. Mann, N. Ryder, et al., “Language Models Are Few-Shot Learners,” Proc. NeurIPS, 2020. DOI: Not available.

10.2 Books

1. D. Jurafsky and J. H. Martin, Speech and Language Processing, 3rd ed. Pearson, 2023. DOI: Not available.
2. S. Bird, E. Klein, and E. Loper, Natural Language Processing with Python. O’Reilly Media, 2009. DOI: Not available.

10.3 Framework & Library Documentation

1. LangChain Documentation. Available: <https://python.langchain.com/> DOI: Not available.
2. LangGraph Documentation. Available: <https://reference.langchain.com/python/langgraph/graphs/> DOI: Not available.
3. Streamlit Documentation. Available: <https://docs.streamlit.io/> DOI: Not available.
4. SentenceTransformers Documentation (MiniLM-L6-v2). Available: <https://www.sbert.net/> DOI: Not available.
5. FAISS Documentation (Facebook AI Similarity Search). Available: <https://faiss.ai/> and <https://github.com/facebookresearch/faiss> DOI: Not available.
6. Redis Official Documentation. Available: <https://redis.io/docs/> DOI: Not available.
7. OpenAI API Documentation (GPT-4 family, including GPT-4o-mini). Available: <https://platform.openai.com/docs> DOI: Not available.

10.4 Tools & Technologies

1. Python Software Foundation, Python Documentation. Available: <https://docs.python.org/3/> DOI: Not available.
2. NumPy Documentation. Available: <https://numpy.org/doc/> DOI: Not available.
3. Python Pickle Serialization Documentation. Available: <https://docs.python.org/3/library/pickle.html> DOI: Not available.
4. PDFMiner Documentation. Available: <https://pypi.org/project/pdfminer/> DOI: Not available.
5. Tesseract OCR Documentation. Available: <https://github.com/tesseract-ocr/tesseract> DOI: Not available.
6. 10.5 Online Learning Resources (Non-Citable)
7. DeepLearning.AI – NLP Specialization.
Stanford CS224N – NLP with Deep Learning.
HuggingFace Transformers Course.

10.6 Additional Articles

1. Meta AI Research – Vector Databases and Similarity Search. Available: <https://ai.facebook.com/> DOI: Not available.
2. NVIDIA Developer – Retrieval-Augmented Generation Articles. Available: <https://developer.nvidia.com/> DOI: Not available.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all those who contributed, directly or indirectly, to the successful completion of this project titled “Agentic Document Question Answering System”.

First and foremost, I am deeply thankful to my S.T Khajekar for their valuable guidance, constant encouragement, and constructive feedback throughout the development of this project. Their insights and suggestions played a crucial role in shaping the technical direction and improving the overall quality of the work.

I would also like to thank the Head of the Department and all the faculty members of the department for providing the necessary academic environment, resources, and support required to carry out this project successfully.

I extend my sincere appreciation to the developers and research community behind open-source technologies and frameworks such as Python, SentenceTransformers, FAISS, Redis, LangChain, LangGraph, Streamlit, and OpenAI, which formed the technical foundation of this system. The availability of high-quality research papers, documentation, and learning resources greatly assisted in understanding and implementing advanced concepts.